Tensor Omics: Core Preprocessing, Analysis Pipeline, and Geometric Tools for Semantic Interpretation

Bobito et al.

April 11, 2025

1 Overview

This document outlines the sequential pipeline used in Tensor Omics for preprocessing, vector space construction, outlier detection, and geometric analysis of gene expression data.

2 Normalization of Expression Values

Raw gene expression data is normalized in the following steps:

1. Division by gene-wise standard deviation for scale normalization.
2. Quantile normalization across samples to reduce global expression biases.
3. Log-transformation: $x \mapsto \log(1 + x)$ to stabilize variance and suppress outliers.
4. For stress response studies, log fold-changes are computed:

$$\Delta_{\text{stress}} = \log(1 + x_{\text{stress}}) - \log(1 + x_{\text{control}})$$

This effectively makes each stress related axis show the fold change in gene expression under stress, e.g. "drought in root divided by control root".

3 Construction of Expression Vector Space

- Biological replicates belonging to the same axis (tissue, stressed tissue, etc.) are averaged post-normalization.
- Each gene is represented as a vector $\vec{p} \in \mathbb{R}^n$, where n is the number of conditions or tissues. Thus n is the number of dimensions of the vector space.
- Expression vectors are stored in Fortran arrays. *Importantly* Fortran is column major and vectors *must* be columns.
- Two K-D Trees are constructed:
  - A raw-space K-D Tree (unnormalized vectors) for Euclidean queries.
  - A normalized-space K-D Tree (unit-length vectors) for cosine similarity and angular queries.
- The first axis is designated the **Anchor Axis (AA)** for visualization.

3.1 Vector Space Storage in Fortran

All expression vectors in Tensor Omics are stored in a central memory structure optimized for performance and access in Fortran. This architecture ensures efficient storage, lookup, and downstream computation.

- **vec_store:** A two-dimensional Fortran array of size $n \times v$, where n is the number of expression dimensions (e.g., tissues or conditions), and v is the number of vectors (e.g., genes). Each column represents one expression vector.
- **cart_tree:** A K-D Tree constructed on the raw (unnormalized) vectors stored in `vec_store`. This structure enables rapid spatial neighborhood queries using Euclidean distance in the original expression space. This is not initialized until step 9.1.
- **sphere_tree:** A second K-D Tree constructed from unit-length normalized vectors derived from `vec_store`. This structure supports directional queries based on cosine similarity or angular distance. This is not initialized until step 9.1.

Together, these data structures allow both magnitude-sensitive and direction-sensitive operations across expression fields.

4 Gene Family Grouping and Centroid Estimation

- Gene families, populations, or functional groups are defined using metadata.
- Family centroids $\vec{o}$ are computed either as:
  - The mean vector of all family members.
  - The mean vector of a family's orthologs only.
- All metadata and vectors can be serialized into `.txdata` binary files.

5 Space Interpolation and Smoothing

To support field-based analysis and ensure consistent representation across expression space, a regular grid is constructed. This grid forms the structural foundation for kernel-based interpolation and vector field smoothing.

5.1 Grid Step Size Estimation

The grid resolution is determined empirically from intra-family expression variation. Specifically, for each gene family, the Euclidean distance between each member's expression vector and the corresponding family centroid is computed. These intra-family distances are aggregated across all families into a single distribution. The grid step size is then set as the **first quartile** (25th percentile) of this distribution.

Note: Since this step requires family membership and centroid information, it must be performed after gene family grouping and centroid estimation (see Section 4).

5.2 Gaussian Kernel Interpolation

After defining the grid, each expression vector contributes to nearby grid points using a Gaussian kernel function centered at its position. For a grid point $\vec{g}$ and an expression vector $\vec{v}$, the kernel weight is defined as:

$$w(\vec{v}, \vec{g}) = \exp\left(-\frac{\|\vec{v} - \vec{g}\|^2}{2\sigma^2}\right)$$

To ensure appropriate spatial influence, the kernel width σ is set to:

$$\sigma = \frac{\text{grid step size}}{\sqrt{2}}$$

This setting ensures the majority of the kernel's weight is concentrated around a single grid cell while still allowing natural smoothing across adjacent cells due to Gaussian decay. Each grid point receives the kernel-weighted average of all expression vectors within range, resulting in a smoothly interpolated vector field.

5.3 Gaussian Kernel Smoothing

Once interpolation is complete, a secondary smoothing step is applied across the grid. Each interpolated vector is recomputed as a Gaussian-weighted average of its neighboring grid vectors, using the same kernel width σ. This step reinforces local coherence, reduces noise, and stabilizes the field across variable-density regions.

6 Empirical Distribution Estimation

To enable robust outlier detection and hypothesis-free exploration, Tensor Omics constructs empirical distributions of core metrics across all gene families.

6.1 Relative Distance Index (RDI)

For each gene family, we define the centroid $\vec{o}$ as the mean of all ortholog vectors $\vec{o}_i$ across species. To assess the magnitude of a gene's divergence from this conserved expression center, we compute the Euclidean distance:

$$d = \|\vec{p} - \vec{o}\|$$

where $\vec{p}$ is the expression vector of the gene being evaluated.

To normalize this distance and enable scale-independent comparison across families, we compute a family-specific scaling factor as the maximum distance between any ortholog vector and the ortholog centroid:

$$d_{\text{scale}} = \max_i \|\vec{o}_i - \vec{o}\|$$

This normalization is linear in the number of orthologs and avoids the quadratic cost of pairwise distance computation. The Relative Distance Index (RDI) is then defined as:

$$\text{RDI} = \frac{\|\vec{p} - \vec{o}\|}{d_{\text{scale}}}$$

In cases where ortholog annotations are unavailable or the family contains only a single ortholog, a statistical smoothing strategy is used to estimate an appropriate scaling factor. Specifically, for each gene family, we calculate:

$$\text{FS}_i = \text{median}_j \left(\|\vec{f}_j - \vec{o}_i\| \right) \quad \text{and} \quad \text{FD}_i = \text{sd}_j \left(\|\vec{f}_j - \vec{o}_i\| \right)$$

where $\vec{f}_j$ are the expression vectors of all family members and $\vec{o}_i$ is the family centroid (mean of all $\vec{f}_j$). A smoothing function $f(\cdot)$, e.g. via LOESS, is fit to the set $\{(\text{FS}_i, \text{FD}_i)\}$ across all families. This yields an empirical mapping from median distance to expected spread. For families lacking orthologs, the smoothed value $f(\text{FS})$ is used as the denominator in:

$$\text{RDI}_{\text{smoothed}} = \frac{\|\vec{p} - \vec{o}\|}{f(\text{FS})}$$

This hybrid strategy ensures that RDI remains well-defined, interpretable, and comparable across evolutionary, functional, and statistical use cases in Tensor Omics.

All RDI values are pooled across all families to build an empirical distribution of relative intra-family divergence. This enables percentile-based filtering of significant shifts (e.g., 95th percentile for outlier detection) without requiring parametric assumptions.

Note: While RDI values are used to identify significant divergence, the unnormalized distances d_i are retained for all downstream geometric computations, such as vector field construction and shift vector analysis.

6.2 Relative Angle Index (RAI)

The angular difference between a vector $\vec{p}$ and its centroid $\vec{o}$ is computed using the cosine formula:

$$\mathrm{RAI} = \cos^{-1}\left(\frac{\vec{p}\cdot\vec{o}}{\|\vec{p}\|\cdot\|\vec{o}\|}\right)$$

This identifies divergence in direction. Thresholds (e.g. 95%) are derived from the pooled distribution of angles.

6.3 Relative Axis Signal (RAS)

For each unit-normalized expression vector $\vec{p}^{\text{norm}}$, we consider its component $\vec{p}_i^{\text{norm}}$ along each axis i (e.g., tissue). For each axis, the 5th percentile across all vectors is computed as the inactivity threshold. A gene is considered neofunctionalized in axis i if:

$$\vec{o}_i^{\text{norm}} < T_i^{(5\%)} \quad \text{and} \quad \vec{p}_i^{\text{norm}} > T_i^{(5\%)}$$

where $\vec{o}^{\text{norm}}$ is the centroid.

6.4 Relative Frobenius Index (RFI)

For any set of vectors $\{\vec{v}_1, \ldots, \vec{v}_n\}$, e.g. gene families, populations, or genes belonging to the same biological process or molecular function group, the vectors are assembled as columns in a matrix $M \in \mathbb{R}^{d\times n}$. The signal strength of this vector-group (matrix) is computed as:

$$\mathrm{RFI} = \frac{\|M\|_F}{\sqrt{n}} = \frac{\sqrt{\sum_{i=1}^{d}\sum_{j=1}^{n} M_{ij}^2}}{\sqrt{n}}$$

This provides a normalized estimate of signal per vector and is used to assess subfield strength.

All empirical distributions are stored in Fortran arrays and used to assign significance thresholds for downstream classification and visualization.

7 Outlier Detection and Shift Vector Calculation

After constructing the vector space and identifying gene families, Tensor Omics detects divergent expression profiles within each family by comparing individual expression vectors to their respective family centroid.

Let $\vec{o} \in \mathbb{R}^n$ denote the centroid of a given family (typically the mean ortholog expression vector), and let $\vec{p}$ denote the expression vector of a candidate paralog. The *raw shift distance* between $\vec{p}$ and the centroid is defined as:

$$d_{po} = \|\vec{p} - \vec{o}\|$$

To determine whether this shift is significant, we normalize the raw distance using the maximum intra-family distance from the centroid. Let $\vec{f}_i$ denote all other expression vectors in the same family. The *Relative Distance Index (RDI)* is calculated as:

$$\mathrm{RDI}_{po} = \frac{d_{po}}{\max_i \|\vec{f}_i - \vec{o}\|}$$

Vectors with an RDI greater than the 95th percentile of the empirical RDI distribution (pooled across all families) are flagged as significant outliers.

Important: Only the raw shift distance d_{po} is used for downstream geometric interpretation, such as vector field construction, trajectory analysis, and clock plot projection. The normalized RDI is used solely for filtering purposes, to robustly identify biologically meaningful divergence across gene families.

Identified outliers are stored as index references in a dedicated Fortran array. For each significant outlier, a corresponding *shift vector* is calculated:

$$\vec{s}_{po} = \vec{p} - \vec{o}$$

These shift vectors are added to the central vector space store ('vec_store') and indexed in the relevant K-D Trees.

8 Tissue Versatility Calculation

- Cosine of angle between expression vector and space diagonal:

$$\cos(\theta) = \frac{\vec{p} \cdot \vec{d}}{\|\vec{p}\| \cdot \|\vec{d}\|}$$

  where $\vec{d} = (1, 1, \ldots, 1)$ is the space diagonal.

9 Clock Plot Projection and Rotation Angle Computation

- All shift vectors are projected into the **Relative Axes Plane (RAP)**, orthogonal to the space diagonal and intersecting the origin.
- The following quantities are calculated for each outlier:
  - Projected shift vector in RAP.
  - Normalized (unit-length) projection.
  - Clock hand vector from origin to projected point.
  - Rotation angle (standardized as clockwise) between projected centroid and outlier.
- Results are stored in a structured Fortran matrix **rap_store**.
  - For each outlier i, store:
    * `Rows 1 to` n: $\vec{s}_i^{\text{RAP}}$
    * `Rows` $n+1$ `to` $2n$: $\vec{s}^{\text{RAP, norm}}$
    * `Rows` $2n+1$ `to` $3n$: $\vec{s}_i^{\text{clock}}$
    * `Row` $3n+1$: rotation angle in radians

9.1 Efficient Vector Lookup Using K-D Trees

To enable fast and scalable querying of gene expression vectors, Tensor Omics employs K-D Trees—an efficient data structure for multidimensional nearest-neighbor searches. The trees are constructed at the very end of the pipeline. Four trees in total, two for all expression and shift vectors, where both are contained in **vec_store** (see 3), and two further for the RAP vectors in **rap_store** (see 9).

Two distinct trees are constructed:

- **cart_tree:** A K-D Tree built from the raw (unnormalized) expression vectors. It supports neighborhood searches in the original Euclidean space, useful for detecting local density patterns or spatial coherence among genes.

- **sphere_tree:** A second K-D Tree constructed from unit-length normalized vectors. This tree enables fast lookup of vectors with similar direction, allowing angular queries based on cosine similarity.

These lookup structures are only built during the final preprocessing stage, prior to interactive analysis. This deferred construction minimizes computational overhead during earlier pipeline steps and ensures consistency with the final state of the vector space. Together, the two trees provide complementary geometric access to the expression landscape.

10 Geometric Tools for Semantic Interpretation

Tensor Omics provides a suite of geometric tools—referred to as *semantic geometry measures*—that support interpretation, subfield detection, and comparative analysis of expression vector fields. These tools operate directly on the vector space and are designed to align with intuitive biological hypotheses.

10.1 Direction-Based Subfield Collector

To detect expression subfields aligned with a biologically meaningful trend, the user defines a direction-of-interest (DOI) vector $\vec{v}_{\text{doi}} \in \mathbb{R}^n$. This may correspond to, for example, increased healing rate, higher oil content, or a linear combination of expression axes.

For each expression vector $\vec{v}_i$, the cosine similarity with the DOI is computed. The method supports two modes, controlled by a Boolean input argument:

- **Normalization enabled:** both vectors are normalized to unit length, and the cosine is computed as:

  $$\cos(\theta_i) = \vec{v}_i^{\text{norm}} \cdot \vec{v}_{\text{doi}}^{\text{norm}}$$

  This mode emphasizes *direction* over magnitude and is suited for analyses where only the semantic trend matters, such as expression coordination in transcription factors or between tissues.

- **Normalization disabled:** the cosine is computed relative to the magnitude of the expression vector:

  $$\cos(\theta_i) = \frac{\vec{v}_i \cdot \vec{v}_{\text{doi}}^{\text{norm}}}{\|\vec{v}_i\|}$$

  This mode retains the influence of vector magnitude and is recommended when working in fold-change spaces (e.g., stress responses), where high expression shifts carry biological meaning.

The resulting cosine values $\cos(\theta_i)$ are used to construct an empirical distribution. Vectors with the strongest projection onto the DOI (e.g., top 5% by cosine value) are selected as a directional subfield. These can then be characterized using the Subfield Descriptor (Section 10.2).

When to Normalize (Direction-Only Interpretation)

Normalization is recommended when:

- The biological question relates to the **direction of change** in expression across conditions.
- Expression magnitude is not meaningful (e.g., low-expression transcription factors).
- Comparing semantic shifts across gene families or tissues.

When Not to Normalize (Magnitude Matters)

Omitting normalization is appropriate when:

- The space encodes **magnitude changes**, e.g., fold-change after stress.
- High-magnitude responses are biologically meaningful, such as upregulation in immune response genes.
- The DOI is used to detect strong response contributors in a specific context.

Selection of Directional Subfield

The cosine values $\cos(\theta_i)$ are collected across all vectors and an empirical distribution is formed. The top 5% (or another percentile threshold) are selected as the directional subfield—those vectors with the strongest alignment (largest projection) onto the DOI.

The selected subset is returned as a list of vector indices and can be further analyzed using subfield descriptors (Section 10.2).

10.2 Subfield Descriptor

Each collected subfield is characterized by aggregating its vectors into a matrix $M \in \mathbb{R}^{n \times k}$, where each column is an expression vector. This matrix is analyzed using the following derived quantities:

- **Flow direction:** The first eigenvector of the covariance matrix $C = MM^T$, representing the dominant direction of expression in the subfield.
- **Rotation vector:** Computed from the antisymmetric component $A = C - C^T$. This vector encodes the local rotation axis and strength.
- **Signal strength:** The Frobenius norm $\|M\|_F$, interpreted as the total signal energy of the field.
- **Directional spread:** The matrix rank of M, indicating how many independent directions are present in the subfield.

For comparability across differently sized subfields, the Frobenius norm can be normalized by the square root of the number of vectors:

$$\text{RFI} = \frac{\|M\|_F}{\sqrt{k}}$$

This Relative Frobenius Index (RFI) provides a density-independent estimate of signal strength.

10.3 Workbench for Interactive Exploration

The Tensor Omics workbench is a lightweight, browser-based environment for exploring the expression space and its geometric structures. It supports:

- 2D and 3D projections (e.g., RAP plane, PCA), with zoom and rotation.
- Efficient filtering via metadata or geometric queries using K-D Trees.
- Visual encoding of vector properties (color, shape, size) by metadata attributes.
- Highlighting of gene families, pathways, or other annotated subsets.

Plots are exportable (e.g., to SVG or PDF), and the current view state can be serialized and restored for reproducibility e.g. by attaching it as request parameters and thus enabling linking to preconfigured interactive plots. This supports sharing, publication, and reproducible workflows.

11 Module Detection via Local Signal Thresholding (LST)

As a simple yet powerful precursor to tensor field–based module detection, Tensor Omics includes a threshold-based method to identify local regions of coherent expression divergence. This approach—referred to as **LST**—operates directly on interpolated shift vector magnitudes, without requiring directionality or flow modeling. It serves as an efficient and broadly applicable first step in identifying biologically meaningful subregions for downstream geometric analysis.

11.1 Threshold Estimation

After the shift field has been constructed and interpolated over the grid, each grid point contains a vector representing the local expression shift. To detect significant divergence, we compute the magnitude (Euclidean norm) of each grid vector and collect the full distribution of these magnitudes across the grid.

A non-parametric background threshold is derived from this distribution by selecting the **5th percentile** of non-zero vector lengths. This provides an adaptive, empirical estimate of the field's background noise level.

11.2 Region Growing Algorithm

Module detection begins at any grid point whose vector magnitude exceeds the threshold. From each such seed point, a circular region is grown iteratively:

- At each radius increment, newly encountered grid points are evaluated.
- Growth continues as long as at least one new point exceeds the magnitude threshold.
- If a full increment produces only sub-threshold points, the region stops growing.

Each grown region is marked as a detected module. Grid points are tracked to avoid duplication. The algorithm proceeds until all grid points exceeding the threshold have been visited.

11.3 Advantages

LST provides a biologically intuitive, parameter-light method to detect modules of divergent expression. It does not rely on directionality, eigenvector extraction, or flow coherence, making it particularly robust in sparse or noisy fields. The detected modules can be further characterized using geometric descriptors (see Section 10.2) or used as inputs to more advanced tensor-based analysis in future phases.

12 Time Series Interpolation and Contextual Smoothing

Tensor Omics applies a two-stage process to prepare time series expression trajectories for vector field analysis. This approach addresses the common problem of sparsely sampled or unevenly spaced time points in transcriptomic time courses and ensures that divergence detection remains sensitive and robust across genes and families.

12.1 Stage 1: Per-Gene Trajectory Interpolation

For each gene, we construct an interpolated expression trajectory using Gaussian kernel-based interpolation. Let the original time points be $t_1, t_2, \ldots, t_n$, with corresponding expression vectors $\vec{e}_{t_i} \in \mathbb{R}^d$. These are interpolated to a denser trajectory with $T = \max(10n, 30)$ total time points, equally spaced over the original time interval.

Interpolation is performed independently per gene, without considering other genes. This preserves the native trajectory shape while increasing temporal resolution. For each interpolated time point t, a Gaussian kernel $K(t, t_i)$ is applied over all original points:

$$K(t, t_i) = \exp\left(-\frac{(t - t_i)^2}{2\sigma_i^2}\right)$$

where σ_i is the local kernel width, chosen adaptively based on sampling density (see below).

12.2 Stage 2: Contextual Smoothing Across Gene Families

To reduce noise and incorporate contextual biological similarity, each interpolated trajectory is smoothed using a Gaussian kernel over time, now also borrowing from ortholog and paralog trajectories.

Each smoothed time point $\vec{e}_t'$ is computed as a weighted average of nearby time points from:

- The same gene (denoted as G)
- Orthologs within the family (denoted as O)
- Paralogs within the family (denoted as P)

The total weight assigned to each group is:

$$\text{Contextual Weights:} \quad \frac{6}{9} \text{ for } G, \quad \frac{2}{9} \text{ for } O, \quad \frac{1}{9} \text{ for } P$$

Within each group, the weight per point is distributed evenly:

$$w_{g_i} = \frac{6}{9|G|}, \quad w_{o_i} = \frac{2}{9|O|}, \quad w_{p_i} = \frac{1}{9|P|}$$

Each point is also assigned a Gaussian kernel weight based on temporal distance from the target time point:

$$K(t, t_i) = \exp\left(-\frac{(t - t_i)^2}{2\sigma_i^2}\right)$$

The final smoothing weight for each point is the product of its contextual weight and kernel weight. The smoothed expression vector is computed as:

$$\vec{e}_t' = \frac{\sum_i w_i K(t, t_i)\vec{e}_{t_i}}{\sum_i w_i K(t, t_i)}$$

12.3 Adaptive Kernel Width

The Gaussian kernel width σ_i is determined locally using symmetric nearest-neighbor spacing:

$$\sigma_i = \begin{cases} t_2 - t_1 & \text{if } i = 1 \\ \frac{1}{2}(t_{i+1} - t_{i-1}) & \text{if } 1 < i < n \\ t_n - t_{n-1} & \text{if } i = n \end{cases}$$

This ensures that smoothing respects the local temporal resolution of the data and avoids arbitrary or global smoothing parameters.

12.4 Implementation Notes

All interpolation and smoothing operations are performed using precomputed Gaussian kernels and group weights. The process is applied prior to shift vector calculation, ensuring that all time-series trajectories are temporally aligned, smooth, and comparable across genes and gene families.

This approach stabilizes vector field construction for downstream clock plots, rotation angle computation, and time-dependent divergence analysis.